

8086 – Trabalhando com arquivos

Quais são as interrupções do msdos responsáveis por manipularem arquivos no 8086?

No MS-DOS para o processador 8086, a manipulação de arquivos é realizada centralizadamente através da interrupção de software **INT 21h**. Conhecida como a principal **API do DOS**, ela utiliza o registrador **AH** para definir qual subfunção (serviço) de arquivo será executada

Historicamente, existem dois métodos para manipular arquivos através da **INT 21h**: o método moderno (via **Handles/Manipuladores de Arquivos**, introduzido no DOS 2.0) e o método legado (via **FCB - File Control Block**, do DOS 1.0).

1. Funções Baseadas em Handles (Recomendado)

Este método utiliza strings com o caminho do arquivo (ASCIIZ) e retorna um identificador numérico de 16 bits (o *handle*).

Valor em AH	Função	Descrição
3Ch	Criar Arquivo	Cria um novo arquivo ou limpa um existente se já houver.
3Dh	Abrir Arquivo	Abre um arquivo existente definindo o modo de acesso (leitura, escrita).
3Eh	Fechar Arquivo	Fecha o arquivo associado ao <i>handle</i> , salvando os buffers pendentes.
3Fh	Ler Arquivo	Lê uma quantidade específica de bytes de um arquivo para a memória.
40h	Escrever Arquivo	Grava dados da memória para o arquivo associado ao <i>handle</i> .
41h	Excluir Arquivo	Remove (deleta) um arquivo do disco com base no seu caminho.
42h	Mover Ponteiro (LSEEK)	Altera a posição de leitura/escrita dentro do arquivo.
56h	Renomear Arquivo	Altera o nome ou move o arquivo de diretório.

2. Funções Baseadas em FCB (Legado / Obsoleto)

Essas funções exigiam que o programador gerenciasse manualmente uma estrutura de dados de 37 bytes na memória chamada *File Control Block*. Não oferecem suporte a subdiretórios (pastas).

Valor em AH	Função	Descrição
0Fh	Abrir Arquivo (FCB)	Abre um arquivo estruturado via bloco de controle.
10h	Fechar Arquivo (FCB)	Fecha o arquivo atualizando o diretório em disco.
14h	Leitura Sequencial	Lê o próximo registro consecutivo do arquivo.
15h	Escrita Sequencial	Grava o próximo registro no arquivo.
16h	Criar Arquivo (FCB)	Cria uma nova entrada de arquivo no diretório raiz.

Exemplo Prático em Assembly 8086

Para abrir um arquivo usando a **INT 21h**, o código padrão costuma seguir esta estrutura:

```
mov ah, 3Dh      ; Subfunção: Abrir arquivo existente
mov al, 0        ; Modo de acesso: 0 = Leitura, 1 = Escrita, 2 = Ambos
mov dx, offset nome ; Endereço da string ASCIIZ com o nome do arquivo
int 21h          ; Chama a interrupção do MS-DOS
jc erro          ; Se o Carry Flag (CF) for ativado, ocorreu um erro
mov handle, ax    ; Se der certo, AX contém o handle do arquivo
```

Para utilizar a função **AH = 42h (LSEEK)**, você precisa passar três informações principais nos registradores:

1. **BX**: O *handle* do arquivo (obtido ao abri-lo).
2. **AL**: O método de movimentação (ponto de partida).
 - 00h: A partir do **início** do arquivo.
 - 01h: A partir da **posição atual** do ponteiro.
 - 02h: A partir do **fim** do arquivo (útil para descobrir o tamanho do arquivo).
3. **CX:DX**: O deslocamento (*offset*) em bytes. Como o 8086 é de 16 bits, o MS-DOS combina dois registradores para permitir arquivos grandes de até 4 GB. **CX** guarda a parte alta (bits mais significativos) e **DX** a parte baixa.

Ao final da execução, se não houver erro, **DX:AX** retornará a nova posição absoluta do ponteiro dentro do arquivo.

Exemplo Prático em Assembly 8086

O exemplo abaixo abre um arquivo chamado **dados.txt**, pula os primeiros 10 bytes dele usando a **42h**, lê os próximos 4 bytes e os exibe na tela.

```
.model small
.stack 100h

.data
filename db "dados.txt", 0    ; Nome do arquivo (String terminada em zero)
buffer   db 4 dup(?)          ; Buffer para armazenar os 4 bytes lidos
handle   dw ?                  ; Variável para guardar o handle do arquivo

.code
main proc
    mov ax, @data
    mov ds, ax

    ; 1. ABRIR O ARQUIVO (Função 3Dh)
    mov ah, 3Dh
    mov al, 0                    ; Modo: Apenas leitura
    lea dx, filename             ; Ponteiro para o nome do arquivo
    int 21h
    jc erro                      ; Se o Carry Flag estiver ativo, houve erro
    mov handle, ax               ; Salva o handle retornado em AX

    ; 2. MOVER O PONTEIRO DO ARQUIVO (Função 42h)
    mov ah, 42h
    mov al, 0                    ; Método 0: Mover a partir do INÍCIO do
arquivo                          ; Handle do arquivo
    mov bx, handle               ; Deslocamento (Parte Alta): 0
    mov cx, 0                    ; Deslocamento (Parte Baixa): 10 bytes
    mov dx, 10                   ; Pula os primeiros 10 bytes
    int 21h                      ; Se CF=1, erro na movimentação
    jc erro                      ; Agora o ponteiro está exatamente no byte 10.
    ; DX:AX agora contém o valor 10 (posição atual).

    ; 3. LER OS PRÓXIMOS 4 BYTES (Função 3Fh)
    mov ah, 3Fh
    mov bx, handle
    mov cx, 4                    ; Quantidade de bytes para ler
    lea dx, buffer               ; Onde salvar os dados lidos
```

```

int 21h
jc erro

; 4. FECHAR O ARQUIVO (Função 3Eh)
mov ah, 3Eh
mov bx, handle
int 21h

; 5. MOSTRAR OS BYTES LIDOS NA TELA (Função 09h da INT 21h)
; Para usar a função 09h, precisamos colocar um '$' no final do buffer
; apenas para fins de exibição na tela.
mov buffer[4], '$'
mov ah, 09h
lea dx, buffer
int 21h
jmp fim

erro:
; Código simplificado de erro (em caso de falha, apenas encerra)
nop

fim:
mov ax, 4C00h ; Encerra o programa retornando ao DOS
int 21h
main endp
end main

```

Truque Comum: Descobrir o tamanho do arquivo

Um uso muito frequente da função 42h é descobrir o tamanho exato de um arquivo em bytes. Para fazer isso, basta mover o ponteiro **0 bytes a partir do fim do arquivo**:

```

mov ah, 42h
mov al, 2 ; Método 2: A partir do FIM do arquivo
mov bx, handle
mov cx, 0 ; Deslocamento 0
mov dx, 0 ; Deslocamento 0
int 21h
; DX:AX agora contém o tamanho exato do arquivo em bytes!

```

Quando usamos `AL=01h` a posição do ponteiro é definida por `CX:DX`?

Sim, a posição final do ponteiro é definida por `CX:DX`, mas com um detalhe crucial: o valor em `CX:DX` funciona como um **deslocamento (offset) relativo** à posição onde o ponteiro já estava, e não como uma posição absoluta.

Quando você usa `AL = 01h` (A partir da posição atual):

- O MS-DOS pega a **posição atual** do ponteiro dentro do arquivo.
- Soma (ou subtrai) o valor de `CX:DX` a essa posição.
- Define o resultado dessa conta como a nova posição do ponteiro.

Como o MS-DOS interpreta o `CX:DX` neste modo:

1. Deslocamento Positivo (Avançar):

Se você quer avançar 50 bytes a partir de onde o ponteiro está agora, basta passar um número positivo:

- `CX = 0000h`
- `DX = 0032h` (50 em hexadecimal)

2. Deslocamento Negativo (Voltar):

O valor em `CX:DX` é tratado como um **número inteiro sinalizado de 32 bits** (em complemento de dois). Isso significa que você pode passar um valor negativo para fazer o ponteiro **voltar** no arquivo. Se você quiser voltar 5 bytes a partir da posição atual, você deve passar o equivalente a `-5` em 32 bits:

- `CX = FFFFh`
- `DX = FFFBh`

O que o MS-DOS retorna após a execução?

Independentemente de ter avançado ou voltado, se a operação for bem-sucedida (Carry Flag `CF = 0`), o MS-DOS atualizará os registradores `DX:AX` com a **nova posição absoluta** (contada a partir do início do arquivo).

Exemplo: Se o ponteiro estava no byte `100`, e você chamou a função com `AL = 01h` e `CX:DX = -5`, a operação colocará o ponteiro no byte `95`. O par de registradores `DX:AX` retornará contendo o valor `95`.

Exemplo: Avançando em Blocos e Voltando o Ponteiro (`AL=01h`)

Imagine que temos um arquivo binário e queremos ler o registro que está na posição `10`, depois pular mais `10` bytes para a frente (avançar) e, por fim, retornar `5` bytes (voltar usando número negativo).

```
.model small
.stack 100h

.data
    filename db "dados.bin", 0
    handle   dw ?
    buffer   db 1 dup(?)

.code
main proc
    mov ax, @data
    mov ds, ax

    ; 1. ABRIR O ARQUIVO (Apenas Leitura)
    mov ah, 3Dh
    mov al, 0
    lea dx, filename
    int 21h
    jc erro
    mov handle, ax

    ; =====
    ; CASO 1: AVANÇAR DE FORMA RELATIVA (Pular 10 bytes a partir do início)
    ; =====
    ; Primeiro, vamos colocar o ponteiro no byte 10 a partir do início (AL=00h)
    mov ah, 42h
    mov al, 0                ; Absoluto (do início)
    mov bx, handle
    mov cx, 0
    mov dx, 10               ; Posição 10
    int 21h

    ; Agora, estando na posição 10, queremos AVANÇAR mais 10 bytes de forma
relativa
    mov ah, 42h
    mov al, 01h              ; RELATIVO À POSIÇÃO ATUAL
    mov bx, handle
    mov cx, 0                ; Deslocamento positivo (parte alta)
    mov dx, 10               ; Deslocamento positivo (parte baixa): +10
bytes
    int 21h
    ; Sucesso! DX:AX agora vale 20 (10 iniciais + 10 do avanço)
```

```

; Lemos o byte da posição 20
mov ah, 3Fh
mov bx, handle
mov cx, 1
lea dx, buffer
int 21h                                ; O DOS lê o byte e avança o ponteiro para 21
automaticamente

; =====
; CASO 2: RETROCEDER DE FORMA RELATIVA (Voltar 5 bytes usando Complemento
de 2)
; =====
; O ponteiro parou na posição 21. Queremos fazê-lo voltar para a posição
16.
; Para voltar 5 bytes, precisamos passar o valor -5 em 32 bits (CX:DX).
; Em hexadecimal de 32 bits (Complemento de 2):
; -5 = FFFFFFFBh -> CX = FFFFh e DX = FFFBh

mov ah, 42h
mov al, 01h                            ; RELATIVO À POSIÇÃO ATUAL
mov bx, handle
mov cx, 0FFFFh                         ; -5 (Parte Alta)
mov dx, 0FFFBh                         ; -5 (Parte Baixa)
int 21h
; Sucesso! DX:AX agora contém 16 (21 atual - 5 do recuo)

; Ler o byte da posição 16
mov ah, 3Fh
mov bx, handle
mov cx, 1
lea dx, buffer
int 21h

; 4. FECHAR O ARQUIVO
mov ah, 3Eh
mov bx, handle
int 21h
jmp fim

erro:
; Trate o código de erro aqui (o código fica em AX)
; Ex: AX = 02h (Arquivo não encontrado), AX = 06h (Handle inválido)
nop

fim:
mov ax, 4C00h
int 21h
main endp
end main

```

Como calcular deslocamentos negativos facilmente?

Para não errar a matemática de binários/hexadecimais sinalizados de 32 bits no Assembly 8086, você pode usar uma diretiva do próprio montador (como TASM ou MASM) para calcular os valores de **CX** e **DX** para você, ou fazer a conta mental do **Complemento de Dois**:

1. Pegue o número positivo em hexa de 32 bits (ex: 5 = 00000005h).
2. Inverta todos os bits (FFFFFFFAh).
3. Some 1 (FFFFFFFBh).
4. Divida o resultado: os 4 primeiros dígitos vão para **CX** (FFFFh) e os 4 últimos para **DX** (FFFBh).